

JOBSHEET 11
LAPORAN PERCOBAAN PRAKTIKUM
LINKED LIST

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

1. Percobaan 1 : Pembuatan Single Linked List

Mahasiswa14.java

```
package Pertemuan11;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    Mahasiswa14() {

    }

    Mahasiswa14(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println(nama + "\t\t" + nim + "\t\t" + kelas + "\t\t" + ipk);
    }
}
```

Node14.java

```
package Pertemuan11;

public class Node14 {
    Mahasiswa14 data;
    Node14 next;

    public Node14(Mahasiswa14 data, Node14 next) {
        this.data = data;
        this.next = next;
    }
}
```

SingleLinkedList14.java

```
package Pertemuan11;

public class SingleLinkedList14 {
    Node14 head;
    Node14 tail;

    boolean isEmpty() {
        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node14 tmp = head;
            System.out.print("Isi Linked List:\n");

            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        }
        else {
            System.out.println("Linked List kosong");
        }
    }

    public void addFirst(Mahasiswa14 input) {
        Node14 ndInput = new Node14(input, null);

        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        }
        else {
            ndInput.next = head;
            head = ndInput;
        }
    }

    public void addLast(Mahasiswa14 input) {
        Node14 ndInput = new Node14(input, null);

        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        }
        else {
            tail.next = ndInput;
            tail = ndInput;
        }
    }

    public void insertAfter(String key, Mahasiswa14 input) {
        Node14 ndInput = new Node14(input, null);
        Node14 temp = head;

        do {
            if (temp.data.nama.equalsIgnoreCase(key)) {
```

```

        ndInput.next = temp.next;
        temp.next = ndInput;
        if (ndInput.next == null) {
            tail = ndInput;
        }
        break;
    }
    temp = temp.next;
} while (temp != null);
}

public void insertAt(int index, Mahasiswa14 input) {
    if (index < 0) {
        System.out.println("Indeks salah");
    }
    else if (index == 0) {
        addFirst(input);
    }
    else {
        Node14 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node14(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
}
}

```

SLLMain14.java

```

package Pertemuan11;

public class SLLMain14 {
    public static void main(String[] args) {
        SingleLinkedList14 sll = new SingleLinkedList14();

        Mahasiswa14 mhs1 = new Mahasiswa14("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa14 mhs2 = new Mahasiswa14("23212201", "Bimon", "2B", 3.8);
        Mahasiswa14 mhs3 = new Mahasiswa14("22212202", "Cintia", "3C", 3.5);
        Mahasiswa14 mhs4 = new Mahasiswa14("21212203", "Dirga", "4D", 3.6);

        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();
    }
}

```

Verifikasi Hasil Percobaan

```
Linked List kosong
Isi Linked List:
Dirga          21212203          4D          3.6

Isi Linked List:
Dirga          21212203          4D          3.6
Alvaro         24212200          1A          4.0

Isi Linked List:
Dirga          21212203          4D          3.6
Cintia         22212202          3C          3.5
Bimon          23212201          2B          3.8
Alvaro         24212200          1A          4.0
```

Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?
2. Jelaskan kegunaan variable temp secara umum pada setiap method!
3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

Jawaban

1. Karena di awal eksekusi, program langsung mengeksekusi `sll.print()`, sedangkan linked list masih belum ada isinya, sehingga menghasilkan “Linked List Kosong”.
2. Pada `SingleLinkedList.java`, ada 3 method yang menggunakan variable, atau lebih tepatnya object temp, yakni `print()`, `insertAfter()`, dan `insertAt()`. Secara umum, object temp pada ketiga method tersebut memiliki kegunaan yang sama, yakni untuk menyimpan sementara object head. Namun, manipulasinya setelah menyimpan sementara itulah yang membedakan antar method. Pada method `print()`, object “tmp”, setelah menampung object head, nantinya akan di-loop untuk menampilkan isi object tersebut selama nilainya tidak null atau selama object tersebut ada isinya. Kemudian, pada `insertAfter()`, object “temp” akan menjadi batu loncatan melalui perulangan hingga program menemukan nama object (mahasiswa) yang sesuai dengan kunci yang diinputkan user dan menyisipkan object baru setelahnya. Terakhir, `insertAt()`, mirip dengan `insertAfter()`, hanya berbeda pada dasar pencariannya, jika `insertAfter()` dasar pencariannya adalah nama mahasiswa, maka `insertAt()` dasar pencariannya adalah index linked list.

3. Perubahan hanya pada SLLMain14.java

```
package Pertemuan11;
import java.util.Scanner;

public class SLLMain14 {

    static int max;
    static Scanner sc = new Scanner(System.in);

    static void menu() {
        System.out.println("\nMenu :\n");
        System.out.println("1. Input Data");
        System.out.println("2. Tampilkan Data");
        System.out.println("0. Keluar\n");
    }

    static void menu1() {
        System.out.println("\nMenu Input Data\n");

        System.out.println("1. Input di awal (index 0)");
        System.out.println("2. Input di akhir (index " + (max - 1) + ")");
        System.out.println("3. Input setelah data tertentu");
        System.out.println("4. Input setelah index tertentu");
        System.out.println("0. Keluar\n");
    }

    static Mahasiswa14 getInput() {

        System.out.println("\nInput data");
        System.out.print("NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Nama   : ");
        String nama = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();
        System.out.print("IPK   : ");
        double ipk = sc.nextDouble();

        Mahasiswa14 mhs = new Mahasiswa14(nim, nama, kelas, ipk);
        //sc.nextLine();
        return mhs;
    }

    public static void main(String[] args) {
        SingleLinkedList14 sll = new SingleLinkedList14();

        // Mahasiswa14 mhs1 = new Mahasiswa14("24212200", "Alvaro", "1A", 4.0);
        // Mahasiswa14 mhs2 = new Mahasiswa14("23212201", "Bimon", "2B", 3.8);
        // Mahasiswa14 mhs3 = new Mahasiswa14("22212202", "Cintia", "3C", 3.5);
        // Mahasiswa14 mhs4 = new Mahasiswa14("21212203", "Dirga", "4D", 3.6);

        /*
        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        */
    }
}
```

```

sll.print();
*/

System.out.println("=== PROGRAM LINKED LIST ===");

int pilihan, subPilihan;

do {
    menu();

    System.out.print("Masukkan pilihan : ");
    pilihan = sc.nextInt();

    switch (pilihan) {
        case 1:

            menu1();

            System.out.print("Masukkan pilihan : ");
            subPilihan = sc.nextInt();

            switch(subPilihan) {
                case 1:
                    sc.nextLine();
                    sll.addFirst(getInput());
                    break;

                case 2:
                    sc.nextLine();
                    sll.addLast(getInput());
                    break;

                case 3:
                    sc.nextLine();
                    System.out.print("Masukkan nama mahasiswa yang ingin anda inputkan data
setelahnya : ");
                    String key = sc.nextLine();
                    Mahasiswa14 mhs1 = getInput();
                    sll.insertAfter(key, mhs1);
                    break;

                case 4:
                    System.out.print("Masukkan index dimana anda ingin inputkan data : ");
                    int idx = sc.nextInt();
                    sc.nextLine();

                    sll.insertAt(idx, getInput());
                    break;

                case 0:
                    break;

                default:
                    System.out.println("\nPilihan tidak valid. Masukkan ulang.");
                    break;
            }

            break;
    }
}

```

```

        case 2:
            sll.print();
            break;

        case 0:
            System.out.println("\nTerima kasih sudah menggunakan layanan kami.\n");
            break;

        default:
            System.out.println("\nPilihan tidak valid. Masukkan ulang.");
            break;
    }

    } while (pilihan != 0);

}
}

```

2. Percobaan 2 : Modifikasi Elemen pada Single Linked List

Mahasiswa14.java

```

package Pertemuan11;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    Mahasiswa14() {

    }

    Mahasiswa14(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println(nama + "\t\t" + nim + "\t\t" + kelas + "\t\t" + ipk);
    }
}

```

Node14.java

```

package Pertemuan11;

public class Node14 {
    Mahasiswa14 data;
    Node14 next;

    public Node14(Mahasiswa14 data, Node14 next) {
        this.data = data;
        this.next = next;
    }
}

```

```
}  
}
```

SingleLinkedList14.java

```
package Pertemuan11;  
  
public class SingleLinkedList14 {  
    Node14 head;  
    Node14 tail;  
  
    boolean isEmpty() {  
        return (head == null);  
    }  
  
    public void print() {  
        if (!isEmpty()) {  
            Node14 tmp = head;  
            System.out.print("Isi Linked List:\n");  
  
            while (tmp != null) {  
                tmp.data.tampilInformasi();  
                tmp = tmp.next;  
            }  
            System.out.println("");  
        }  
        else {  
            System.out.println("Linked List kosong");  
        }  
    }  
  
    public void addFirst(Mahasiswa14 input) {  
        Node14 ndInput = new Node14(input, null);  
  
        if (isEmpty()) {  
            head = ndInput;  
            tail = ndInput;  
        }  
        else {  
            ndInput.next = head;  
            head = ndInput;  
        }  
    }  
  
    public void addLast(Mahasiswa14 input) {  
        Node14 ndInput = new Node14(input, null);  
  
        if (isEmpty()) {  
            head = ndInput;  
            tail = ndInput;  
        }  
        else {  
            tail.next = ndInput;  
            tail = ndInput;  
        }  
    }  
  
    public void insertAfter(String key, Mahasiswa14 input) {
```

```

Node14 ndInput = new Node14(input, null);
Node14 temp = head;

do {
    if (temp.data.nama.equalsIgnoreCase(key)) {
        ndInput.next = temp.next;
        temp.next = ndInput;
        if (ndInput.next == null) {
            tail = ndInput;
        }
        break;
    }
    temp = temp.next;
} while (temp != null);
}

public void insertAt(int index, Mahasiswa14 input) {
    if (index < 0) {
        System.out.println("Indeks salah");
    }
    else if (index == 0) {
        addFirst(input);
    }
    else {
        Node14 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node14(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
}

```

SLLMain14.java

```

package Pertemuan11;
import java.util.Scanner;

public class SLLMain14 {

    static int max;
    static Scanner sc = new Scanner(System.in);

    static void menu() {
        System.out.println("\nMenu :\n");
        System.out.println("1. Input Data");
        System.out.println("2. Tampilkan Data");
        System.out.println("0. Keluar\n");
    }

    static void menu1() {
        System.out.println("\nMenu Input Data\n");

        System.out.println("1. Input di awal (index 0)");
        System.out.println("2. Input di akhir (index " + (max - 1) + ")");
        System.out.println("3. Input setelah data tertentu");
        System.out.println("4. Input setelah index tertentu");
    }
}

```

```

        System.out.println("0. Keluar\n");
    }

    static Mahasiswa14 getInput() {

        System.out.println("\nInput data");
        System.out.print("NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Nama   : ");
        String nama = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();
        System.out.print("IPK   : ");
        double ipk = sc.nextDouble();

        Mahasiswa14 mhs = new Mahasiswa14(nim, nama, kelas, ipk);
        //sc.nextLine();
        return mhs;
    }

    public static void main(String[] args) {
        SingleLinkedList14 sll = new SingleLinkedList14();

        Mahasiswa14 mhs1 = new Mahasiswa14("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa14 mhs2 = new Mahasiswa14("23212201", "Bimon", "2B", 3.8);
        Mahasiswa14 mhs3 = new Mahasiswa14("22212202", "Cintia", "3C", 3.5);
        Mahasiswa14 mhs4 = new Mahasiswa14("21212203", "Dirga", "4D", 3.6);

        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();

        /*
        System.out.println("=== PROGRAM LINKED LIST ===");

        int pilihan, subPilihan;

        do {
            menu();

            System.out.print("Masukkan pilihan : ");
            pilihan = sc.nextInt();

            switch (pilihan) {
                case 1:

                    menu1();

                    System.out.print("Masukkan pilihan : ");
                    subPilihan = sc.nextInt();

                    switch(subPilihan) {

```

```

        case 1:
            sc.nextLine();
            sll.addFirst(getInput());
            break;

        case 2:
            sc.nextLine();
            sll.addLast(getInput());
            break;

        case 3:
            sc.nextLine();
            System.out.print("Masukkan nama mahasiswa yang ingin anda inputkan data
setelahnya : ");
            String key = sc.nextLine();
            Mahasiswa14 mhs1 = getInput();
            sll.insertAfter(key, mhs1);
            break;

        case 4:
            System.out.print("Masukkan index dimana anda ingin inputkan data : ");
            int idx = sc.nextInt();
            sc.nextLine();

            sll.insertAt(idx, getInput());
            break;

        case 0:
            break;

        default:
            System.out.println("\nPilihan tidak valid. Masukkan ulang.");
            break;
    }

    break;

case 2:
    sll.print();
    break;

case 0:
    System.out.println("\nTerima kasih sudah menggunakan layanan kami.\n");
    break;

default:
    System.out.println("\nPilihan tidak valid. Masukkan ulang.");
    break;
}

} while (pilihan != 0);
*/

System.out.println("Data index 1 : ");
sll.getData(1);

System.out.println("Data mahasiswa AN Bimon berada pada index : " + sll.indexOf("bimon"));
System.out.println();

sll.removeFirst();

```

```

        sll.removeLast();
        sll.print();
        sll.removeAt(0);
        sll.print();
    }
}

```

Verifikasi Hasil Percobaan

```

Data index 1 :
Cintia      22212202      3C      3.5
Data mahasiswa AN Bimon berada pada index : 2

Isi Linked List:
Cintia      22212202      3C      3.5
Bimon      23212201      2B      3.8

Isi Linked List:
Bimon      23212201      2B      3.8

```

Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!
2. Jelaskan kegunaan kode dibawah pada method remove

```

1  temp.next = temp.next.next;
2  if (temp.next == null) {
3      tail = temp;
4  }

```

Jawaban

1. Agar setelah data ditemukan, perulangan berhenti dan tidak mencari dan menghapus lagi.
2. Seperti yang telah dikerjakan pada ASD sebelumnya, kode tersebut mengubah temp.next, yakni node setelah index yang ingin dihapus - 1 dengan temp.next.next, yakni node setelah node yang ingin dihapus.

temp → temp.next → temp.next.next

setelah kode temp.next = temp.next.next

temp → temp.next.next